

TARİH DEDEKTİFİ

Kapsamlı Teknik Dokümantasyon ve Yazılım Mimarisi Raporu

1. Proje Hakkında Genel Bakış

Tarih Dedektifi, oyuncuların kendilerini tarihin en kritik ve gizemli anlarının tam merkezinde buldukları, metin tabanlı ve yapay zeka destekli bir mobil rol yapma (RPG) oyunudur. Geleneksel çizgisel hikaye anlatımının dışına çıkan bu yapıda, her seçim hikayenin akışını, karakterlerin oyuncuya olan yaklaşımını ve oyunun sonunu doğrudan etkiler. Proje; üretken yapay zeka teknolojileri, dinamik görsel üretimi ve modern mobil teknolojilerin entegrasyonu ile zenginleştirilmiş sürükleyici bir deneyim sunmayı hedefler.

2. Teknolojik Altyapı ve Sistem Mimarisi

Uygulama, esnek, modüler ve yüksek performanslı bileşenlerden oluşan katmanlı bir yazılım mimarisine sahiptir. Kullanılan temel kütüphaneler ve sistem katmanları aşağıda detaylandırılmıştır:

Bileşen / Katman	Kullanılan Teknoloji / Paket	Sistemdeki Görevi ve Amacı
Ana Çerçeve	Flutter & Dart (Aydınlık Tema)	Çapraz platform destekli, yüksek performanslı arayüz katmanı.
Yapılandırılmış AI	Google Gemini API (gemini-2.5-flash)	Oyunun yönetimini üstlenen, kararları işleyen ve kurguyu yöneten ana motor.
Görsel Üretimi	Pollinations.ai (Flux Model)	Hikayenin geçtiği sahnelere ait anlık ve dinamik görsellerin üretimi.
Sohbet Motoru	dash_chat_2	Oyuncu ile Yapay Zeka rehberi arasındaki mesajlaşma akış mimarisi.
Durum Yönetimi	Riverpod Notifier (Altyapı)	İş mantığı (Business Logic) ile UI katmanının birbirinden izole edilmesi.
Veri Kalıcılığı	shared_preferences	Can, envanter ve mesaj geçmişinin yerel cihazda şifreli/yapılandırılmış kaydı.
Erişilebilirlik / Ses	flutter_tts & speech_to_text	Hikayenin sesli okunması (Metin Okuma) ve mikrofon tabanlı sesli yanıt girişi.
Görsel Efektler	shimmer & animated_text_kit	Görsel yükleme esnasındaki banyo efekti ve daktilo yazı animasyonu.
Dokümantasyon	pdf & printing	Oyun sonu veya oturum esnasında diyalogların vaka raporu olarak dışa aktarımı.

3. Temel Özellikler ve Oyun Mekanikleri

3.1. Yapay Zeka Oyun Motoru ve JSON Haberleşme Protokolü

Oyunun tüm akışı, Gemini API'ye entegre edilmiş katı bir Sistem Talimatı (System Instruction) şablonuna dayanır. Yapay zeka serbest metin üretmek yerine, istemci (mobil uygulama) tarafında güvenli ayrıştırılabilecek (parse edilebilecek) ve durum yönetimini tetikleyebilecek mutlak bir JSON yapısı döndürür. İletişim protokolü şu şekildedir:

```
{
  "story": "Olay örgüsünü anlatan ana metin bloğu...", // String
  "image_prompt": "Scene description for generative image models...", // String
  (EN)
  "options": ["Seçenek A", "Seçenek B"], // List<String>
  "item_found": "Eşya ismi veya null", // String?
  "health_change": -15, // Integer
  "is_game_over": false // Boolean
}
```

3.2. Dinamik Can (Health) ve Akıllı İpucu (Hint) Sistemi

Oyuncu maceraya 100 can puanı ile başlar. Yapay zeka motoru, oyuncunun kararlarının tehlike düzeyine göre JSON içerisindeki health_change parametresini günceller. Oyuncu kritik anlarda tıkanıldığında, üst barda yer alan ampul butonunu kullanarak **İpucu Sistemi**'ni tetikleyebilir. Bu işlem oyuncuya 10 can puanına mal olur ve yapay zekaya üstü kapalı, gizemli bir yönlendirme yaptırır. Can değeri 10 ve altında ise sistem bu özelliğin kullanımını engeller.

3.3. Dinamik Envanter ve Rastgele Tetiklenen Mini Bulmacalar

Yapay zeka hikaye akışında oyuncuya bir nesne verdiğinde, bu nesne anlık olarak ayrıştırılarak oyuncunun sırt çantasına (envanterine) eklenir. Envanterde iki veya daha fazla nesne biriktiğinde, oyun motoru arka planda çalışan matematiksel bir şans algoritmasıyla native bir **Mini Bulmaca** penceresi açabilir. Oyuncudan elindeki ipuçlarını mantıksal olarak birleştirmesi istenir; girilen cevap yeni bir girdi olarak yapay zekaya gönderilir ve kurgu derinleştirilir.

3.4. Gelişmiş UI/UX Animasyonları ve Sahne Geçişleri

- **Daktilo Efekt:** Yapay zekadan gelen hikaye metinleri ekranda aniden belirmek yerine, retro dedektiflik atmosferini desteklemek amacıyla saniyede belirli bir karakter hızıyla harf harf ekrana yazılır.
- **Fotoğraf Banyo (Shimmer) Efekt:** Yapay zeka tarafından sağlanan İngilizce prompt, Pollinations API'sine gönderildiğinde görselin indirilme sürecinde aydınlık tonda tasarlanmış bir Shimmer animasyonu devreye girer. Ekranda "📷 Sahne oluşturuluyor..." animasyonu oynatılarak kullanıcının bekleme deneyimi optimize edilir.

3.5. Yerel Veri Kalıcılığı (Save/Load)

Oyunun kopmaması ve uzun soluklu bir deneyim sunması adına, her başarılı mesajlaşma döngüsünün ardından oyuncunun can durumu, envanteri, aktif seçenekleri ve diyalog geçmişi SharedPreferences katmanında ilgili vaka başlığı (topic) altında asenkron olarak diske yazılır. Uygulama yeniden başlatıldığında aynı konu girilirse oyun kaldığı yerden sıfır kayıpla devam eder.

4. Proje Klasör Yapısı (Folder Structure)

Temiz kod (Clean Code) standartlarına ve sorumlulukların ayrılması (Separation of Concerns) prensibine uygun olarak tasarlanan dizin yapısı şu şekildedir:

```
lib/  
|  
├─ main.dart          # Uygulama giriş noktası, WelcomeScreen ve tema  
bağlayıcı.  
├─ theme.dart         # Aydınlık, renkli ve dinamik tema konfigürasyonu.  
├─ game_screen.dart   # DashChat entegrasyonu, oyun mekanikleri ve AI  
haberleşmesi.  
└─ pdf_service.dart   # Raporlama modülü, PDF oluşturma ve paylaşım servisi.
```

5. Güvenlik Notu ve API Yönetimi

ÖNEMLİ GÜVENLİK UYARISI: Mevcut mimaride kolay test edilebilirlik amacıyla API anahtarı kod içerisinde statik olarak yer almaktadır. Üretim (Production) ortamına geçişte, kaynak kodun tersine mühendislik (reverse engineering) yöntemleriyle çözülerek anahtarın ele geçirilmesini önlemek amacıyla kesinlikle flutter_dotenv paketi kullanılmalı veya istekler güvenli bir backend/mikroservis mimarisi (Örn: Nexus BaaS) üzerinden tünellenmelidir.

6. Kurulum ve Çalıştırma Kılavuzu

Projeyi yerel ortamınızda ayağa kaldırmak için aşağıdaki adımları sırasıyla uygulayınız:

1. Proje deposunu yerel bilgisayarınıza kopyalayın.
2. Terminal üzerinden proje ana dizinine giderek gerekli paketleri yüklemek için bağımlılıkları çalıştırın:

```
flutter pub get
```

3. Cihaz bağlantınızı kontrol edin ve uygulamayı debug modunda başlatın:

```
flutter run
```

7. Gelecek Yol Haritası ve Geliştirme Planı

- Hikaye JSON yapısından dönecek duygu durumuna göre (Gergil, Heyecanlı, Gizemli) arka planda çalan müzik tonunun dinamik olarak değişmesi (audioplayers entegrasyonunun genişletilmesi).

- Çözölen vakaların puanlanarak küresel bir skor tabelasında (Leaderboard) listelenmesini sağlayacak bulut veritabanı entegrasyonu.

Tarih Dedektifi Yazılım Geliştirme Departmanı Resmi Teknik Raporu © 2026